



MANUAL TECNICO

USAC PROCESSING DATA

Emily Maritza Tepeu Guacamaya

202402955

Práctica 2

IPC1_F

TABLA DE CONTENIDO

INTRODUCCIÓN	4
REQUISITOS	5
Requisitos Mínimos del Sistema	5
Sistema Operativo	5
Entorno de Desarrollo Integrado (IDE)	5
Java Runtime Environment (JRE)	5
Memoria RAM	5
Espacio en Disco	5
Librerías Externas	5
CONFIGURACIÓN DEL ENTORNO	6
Instalación del JDK	6
Configuración del IDE	6
Ejecución	6
¿QUÉ ES EL MODELO VISTA CONTROLADOR?	7
Modelo	7
Vista	7
Controlador	7
ESTRUCTURA DEL CÓDIGO	8
Paquete Vista	8
PantallaPVista	¡Error! Marcador no definido.
OrdenamientoVista	¡Error! Marcador no definido.
Ordenamientos	¡Error! Marcador no definido.
Paquete Modelo	14
PantallaPModelo	¡Error! Marcador no definido.
OrdenamientoModelo	¡Error! Marcador no definido.
BurbujaModelo	¡Error! Marcador no definido.
InsertsortModelo	¡Error! Marcador no definido.
MergesortModelo	¡Error! Marcador no definido.
QuicksortModelo	¡Error! Marcador no definido.
SelectsortModelo	¡Error! Marcador no definido.
ShellsortModelo	¡Error! Marcador no definido.
Paquete Controlador	19

PantallaPControlador _____ ¡Error! Marcador no definido.
OrdenamientoControlador _____ ¡Error! Marcador no definido.
BurbujaControlador _____ ¡Error! Marcador no definido.
InsertsortControlador _____ ¡Error! Marcador no definido.
MergesortControlador _____ ¡Error! Marcador no definido.
QuicksortControlador _____ ¡Error! Marcador no definido.
SelectsortControlador _____ ¡Error! Marcador no definido.
ShellsortControlador _____ ¡Error! Marcador no definido.

CONSIDERACIONES TÉCNICAS _____ 1

¿Qué formato debe tener el archivo csv para ser procesado? _____ 1
¿Qué pasa si el archivo csv no cumple con el formato requerido? _____ 1
¿Puedo ordenar los datos sin cargar un archivo primero? _____ 1
¿Cómo cambio la velocidad del ordenamiento? _____ ¡Error! Marcador no definido.
¿Dónde puedo ver el reporte PDF después de generarlo? _____ 1

INTRODUCCIÓN

El sistema desarrollado permite administrar empleados, clientes, automóviles, servicios y repuestos, incluyendo funcionalidades como carga masiva de datos, control de inventario, seguimiento del progreso de vehículos mediante hilos, generación de facturas, y elaboración de reportes en PDF con gráficos estadísticos. Además, se ha trabajado con la persistencia de datos mediante serialización de objetos en archivos binarios.

Este documento técnico describe la estructura general del sistema, los métodos implementados, los diagramas utilizados, y los aspectos técnicos considerados para garantizar el correcto funcionamiento del software.

REQUISITOS

Requisitos Mínimos del Sistema

Antes de ejecutar el programa, asegúrese de cumplir con los siguientes requisitos:

Sistema Operativo

- Windows 7 o superior
- macOS 10.12 (Sierra) o superior
- Linux (distribuciones compatibles con Java)

Entorno de Desarrollo Integrado (IDE)

- NetBeans: <https://netbeans.apache.org/>
- Eclipse: <https://www.eclipse.org/downloads/>
- IntelliJ IDEA: <https://www.jetbrains.com/idea/>

Java Runtime Environment (JRE)

- Versión: Java 8 o superior
- Descarga: Puedes obtener el JRE desde Oracle JDK u OpenJDK

Memoria RAM

- Mínimo: 4 GB
- Recomendado: 8 GB o más para un mejor rendimiento

Espacio en Disco

- Mínimo: 500 MB de espacio libre para almacenar el programa y los datos generados.

Librerías Externas

- **iTextPDF**: se usa para la generación de reportes en formato PDF.
- **JCommon**: se usa para poder ejecutar el JFreeChart correctamente.
- **JFreeChart**: es un marco de software que permite la creación de gráficas complejas de una forma sencilla.

CONFIGURACIÓN DEL ENTORNO

Instalación del JDK

- Descargar e instalar el JK desde el sitio oficial.
- Configurar las variables JAVA_HOME para apuntar la ruta de instalación del JDK.
- Asegurarse de que el comando java y javadk estén disponibles en la terminal.

Configuración del IDE

- Importar el proyecto como un proyecto Java estándar.
- Asegurarse de que el JDK correcto esté configurado en las opciones del IDE.

Ejecución

- Para ejecutar el programa deberá ir a la carpeta del archivo y buscar la carpeta dist, en este habrá un archivo .jar llamado Taller, haga doble click y este iniciará el programa.

¿QUÉ ES EL MODELO VISTA CONTROLADOR?

El Modelo-Vista-Controlador (MVC) es un patrón de diseño arquitectónico utilizado en el desarrollo de software para separar la lógica de negocio, la interfaz de usuario y la gestión de las interacciones del usuario. Este patrón organiza el código en tres componentes principales:

Modelo

Representa la lógica de negocio y los datos de la aplicación. Se encarga de gestionar la información, realizar operaciones y aplicar reglas de la aplicación. En este caso permite generar el código de los ordenamientos y como se adapta cada uno a los diferentes casos.

Vista

Es la interfaz de usuario (UI) que muestra la interfaz al usuario, así mismo le permite visualizar las gráficas creadas, así mismo ordenarlas y visualizar el paso a paso del ordenamiento. La vista no contiene lógica, solo se encarga de presentar los datos proporcionados por el modelo. En la aplicación, las clases como *BurbujaModelo*, *ShellsortModelo* y *VelocidadModelo* son ejemplos de vistas.

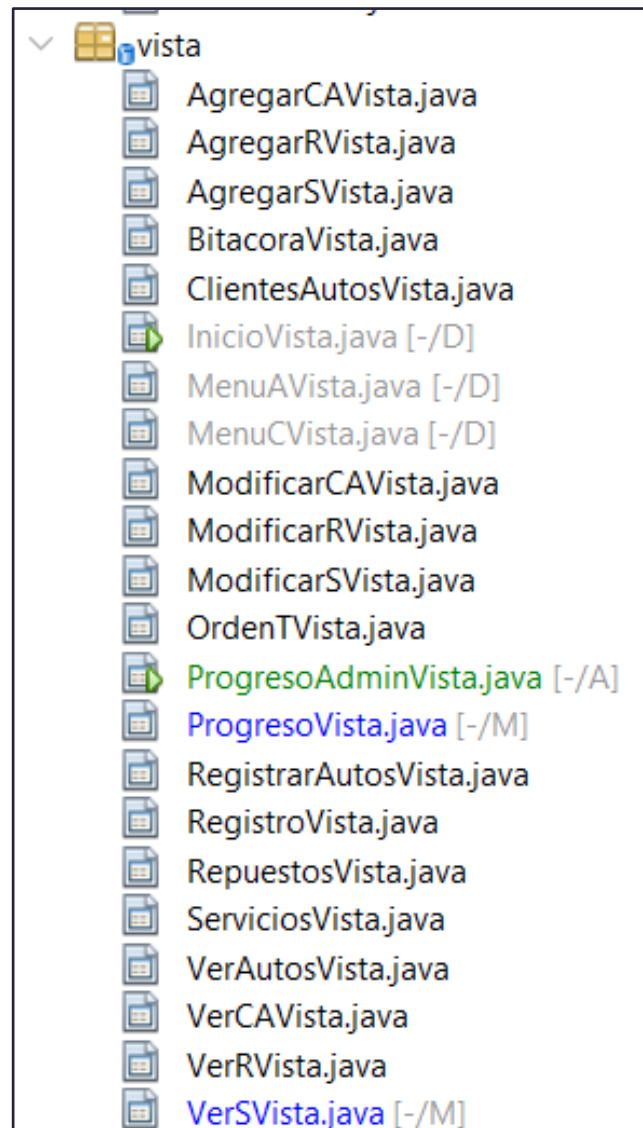
Controlador

Actúa como intermediario entre el modelo y la vista. Recibe las interacciones del usuario desde la vista, procesa las solicitudes (usando el modelo) y actualiza la vista con los resultados. En la aplicación, los controladores como *BurbujaControlador*, *ShellshortControlador*, *QuicksortControlador* y *ReportesControlador*, gestionan la lógica de las operaciones y coordinan las actualizaciones entre el modelo y la vista.

ESTRUCTURA DEL CÓDIGO

Paquete Vista

Es el encargado de mostrar la interfaz para crear las gráficas y capturar las interacciones del usuario. Así mismo se muestra el IDE de la ventana, donde se muestra el diseño en código. En este paquete podemos encontrar las siguientes clases:



IncioVista

En esta clase se puede iniciar sesión como administrador, cliente o mecanico, con las credenciales: "adminipc1f", "mecanicoipc1f" o bien, registrar una cuenta y sus credenciales.

```
//GETTERS
public javax.swing.JButton getBotonIniciar() {
    return botonIniciar;
}

public javax.swing.JButton getBotonRegistro() {
    return botonRegistro;
}

public javax.swing.JPasswordField getCampoContraseña() {
    return campoContraseña;
}

public javax.swing.JTextField getCampoUsuario() {
    return campoUsuario;
}
```

RegistroVista

En este clase se configuraron los setters para hacer que funcionen correctamente los botones.

```
//Getters
public javax.swing.JRadioButton getBotonNormal() {
    return botonNormal;
}

public javax.swing.JRadioButton getBotonOro() {
    return botonOro;
}

public javax.swing.JTextField getCampoContra() {
    return campoContra;
}

public javax.swing.JTextField getCampoDPI() {
    return campoDPI;
}

public javax.swing.JTextField getCampoNombre() {
    return campoNombre;
}

public javax.swing.JTextField getCampoUsuario() {
    return campoUsuario;
}

public javax.swing.JButton getBotonAceptar() {
    return botonAceptar;
}
```

BitacoraVista

Esta clase se ejecuta al momento de iniciar el programa.

```
public BitacoraVista() {
    initComponents();
    setTitle("Bitácora del Sistema");
    setDefaultCloseOperation(javax.swing.WindowConstants.DO_NOTHING_ON_CLOSE);
}
```

MenuCVista

En esta clase se realizan los siguientes getters para su funcionamiento:

```
//Getters
public javax.swing.JButton getBotonCerrar() {
    return botonCerrar;
}

public javax.swing.JButton getBotonClientes() {
    return botonClientes;
}

public javax.swing.JButton getBotonProgreso() {
    return botonProgreso;
}

public javax.swing.JButton getBotonReporte() {
    return botonReporte;
}

public javax.swing.JButton getBotonRepuestos() {
    return botonRepuestos;
}

public javax.swing.JButton getBotonServicios() {
    return botonServicios;
}
```

MenuAVista

En esta clase se realizan los siguientes getters para su funcionamiento:

```
//Getters
public javax.swing.JButton getBotonAutos() {
    return botonAutos;
}

public javax.swing.JButton getBotonCerrar() {
    return botonCerrar;
}

public javax.swing.JButton getBotonFacturas() {
    return botonFacturas;
}

public javax.swing.JButton getBotonProgreso() {
    return botonProgreso;
}

public javax.swing.JButton getBotonRegistrar() {
    return botonRegistrar;
}
```

RepuestosVista

En esta clase se realizan los siguientes getters para su funcionamiento:

```
public javax.swing.JButton getBotonAgregar() {
    return botonAgregar;
}

public javax.swing.JButton getBotonModificar() {
    return botonModificar;
}

public javax.swing.JButton getBotonEliminar() {
    return botonEliminar;
}

public javax.swing.JButton getBotonVer() {
    return botonVer;
}

public javax.swing.JButton getBotonRegresar() {
    return botonRegresar;
}
```

AgregarRVista

En esta clase se hace uso del drag and drop con el siguiente código para que pueda aceptar solamente archivos con extensión .tmr

```
public boolean importData(TransferSupport support) {
    try {
        java.util.List files = (java.util.List) support.getTransferable().getTransferData(DataFlavor.javaFileListFlavor);
        java.io.File file = (java.io.File) files.get(0);

        if (!file.getName().endsWith(".tmr")) {
            javax.swing.JOptionPane.showMessageDialog(null, "EL ARCHIVO DEBE TENER EXTENSION .tmr", "ERROR", javax.swing.J
            return false;
        }

        java.util.Scanner scanner = new java.util.Scanner(file);
        if (scanner.hasNextLine()) {
            String linea = scanner.nextLine();
            String[] partes = linea.split("-");

            if (partes.length != 5) {
                javax.swing.JOptionPane.showMessageDialog(null, "SE REQUIEREN 5 DATOS SEPARADOS POR '-'", "ERROR", javax.s
                return false;
            }

            campoNombre.setText(partes[0]);
            campoMarca.setText(partes[1]);
            campoModelo.setText(partes[2]);
            campoExistencias.setText(partes[3]);
            campoPrecio.setText(partes[4]);

            return true;
        }
    }
}
```

ModificarRVista

En esta clase se realizan los siguientes getters para su funcionamiento:

```
public javax.swing.JTextField getCampoNombre() {
    return campoNombre;
}

public javax.swing.JTextField getCampoMarca() {
    return campoMarca;
}

public javax.swing.JTextField getCampoModelo() {
    return campoModelo;
}

public javax.swing.JTextField getCampoExistencias()
    return campoExistencias;
}

public javax.swing.JTextField getCampoPrecio() {
    return campoPrecio;
}

public javax.swing.JButton getBotonAceptar() {
    return botonAceptar;
}

public javax.swing.JButton getBotonRegresar() {
    return botonRegresar;
}
```

VerRVista

En esta clase se realizan los siguientes getters para su funcionamiento:

```
//GETTERS
public javax.swing.JTable getTabla() {
    return tabla;
}

public javax.swing.JButton getBotonRegresar() {
    return botonRegresar;
}
```

ProgresoVista

En esta clase se realizan los siguientes getters para su funcionamiento:

```
//GETTERS
public javax.swing.JProgressBar getColaEspera() {
    return colaEspera;
}

public javax.swing.JProgressBar getEnServicio() {
    return enServicio;
}

public javax.swing.JProgressBar getListoEntrega() {
    return listoEntrega;
}

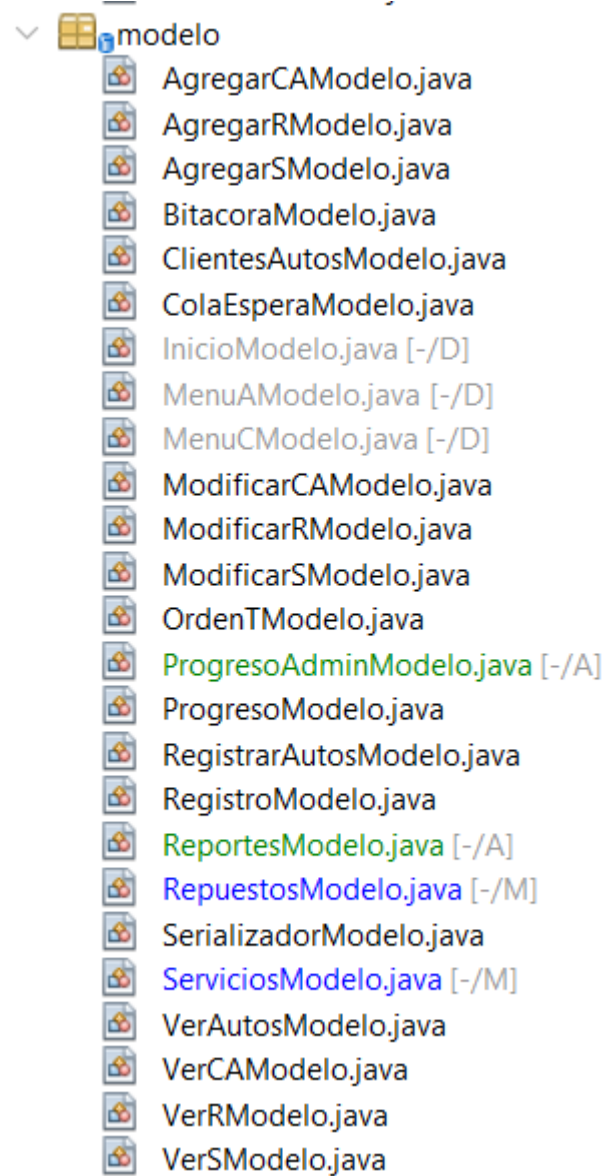
public javax.swing.JButton getBotonAñadir() {
    return botonAñadir;
}

public javax.swing.JButton getBotonProcesar() {
    return botonProcesar;
}

public javax.swing.JButton getBotonRegresar() {
    return botonRegresar;
}
```

Paquete Modelo

La mayoría de las clases creadas que cuentan con vista y controlador, no cuentan con modelo ya que no fue necesario para su funcionamiento, las clases en este paquete son:



De las cuales solo las siguientes tienen modelo:

InicioModelo

En esta clase se validan las credenciales:

```
public class InicioModelo {
    public static final String ADMIN_USER = "adminipclf";
    public static final String ADMIN_PASS = "adminipclf";
    public static final String MECANICO_USER = "mecanicoipclf";
    public static final String MECANICO_PASS = "mecanicoipclf";
}
```

BitacoraModelo

En esta clase se coloca el formato o estructura de la tabla con los datos que debe tener:

```
public static class RegistroBitacora {
    private String marcaTemporal;
    private String usuario;
    private String accion;
    private String resultado;
    private String detalles;

    public RegistroBitacora(String usuario, String accion, String resultado, String detalles) {
        SimpleDateFormat sdf = new SimpleDateFormat("dd/MM/yyyy HH:mm:ss");
        this.marcaTemporal = sdf.format(new Date());
        this.usuario = usuario;
        this.accion = accion;
        this.resultado = resultado;
        this.detalles = detalles;
    }

    public String getMarcaTemporal() { return marcaTemporal; }
    public String getUsuario() { return usuario; }
    public String getAccion() { return accion; }
    public String getResultado() { return resultado; }
    public String getDetalles() { return detalles; }
}
```

ColaEsperaModelo

La cola de espera tiene un funcionamiento para funcionar correctamente en el progreso:

```
public ColaEsperaModelo(int capacidad) {
    this.capacidad = capacidad;
    this.cola = new String[capacidad][5]; // placa, marca, modelo, servicio, tipoCliente
    this.frente = 0;
    this.fin = -1;
}

public static ColaEsperaModelo getInstance(int capacidad) {
    if (instancia == null) {
        instancia = cargarDatos();
        if (instancia == null || instancia.capacidad != capacidad) {
            instancia = new ColaEsperaModelo(capacidad);
        }
    }
    return instancia;
}

public static void guardarDatos() {
    if (instancia != null) {
        serializador.guardarDatos(ARCHIVO_DATOS, instancia);
    }
}
```

ProgresoModelo

En esta clase va la estructura por defecto para obtener el vehiculo del proceso y el tipo de cliente.

```
public boolean agregarVehiculoProceso(String placa, String marca, String modelo, String servicio, String usuario) {
    if (contadorVehiculos >= vehiculosEnProceso.length) {
        return false;
    }

    String tipoCliente = obtenerTipoCliente(usuario);
    vehiculosEnProceso[contadorVehiculos][0] = placa;
    vehiculosEnProceso[contadorVehiculos][1] = marca;
    vehiculosEnProceso[contadorVehiculos][2] = modelo;
    vehiculosEnProceso[contadorVehiculos][3] = servicio;
    vehiculosEnProceso[contadorVehiculos][4] = tipoCliente;
    colaEsperaModelo.encolarConPrioridad(placa, marca, modelo, servicio, tipoCliente);
    contadorVehiculos++;
    guardarDatos();
    return true;
}

private String obtenerTipoCliente(String usuario) {
    ClientesAutosModelo.Cliente cliente = ClientesAutosModelo.buscarClientePorUsuario(usuario);
    return cliente != null ? cliente.getTipoCliente() : "NORMAL";
}
```

RegistrarAutosModelo

En esta clase se han creado las carpetas para guardar los datos con serialización.

```
public class RegistrarAutosModelo implements Serializable {
    private static final long serialVersionUID = 1L;
    private static final String ARCHIVO_DATOS = "registro_autos.dat";
    private static final String CARPETA_IMAGENES = "imagenes_autos";
    private static final SerializadorModelo serializador = new SerializadorModelo();
    private static RegistrarAutosModelo instancia;

    private Auto[] autosRegistrados;
    private int capacidad;
    private int cantidadAutos;

    private RegistrarAutosModelo() {
        this.capacidad = 100; //CAPACIDAD INICIAL
        this.autosRegistrados = new Auto[capacidad];
        this.cantidadAutos = 0;
        crearCarpetasSiNoExisten();
    }
}
```

ReportesModelo

En esta clase se realizó la estructura para poder generar los reportes en pdf con sus respectivas especificaciones y llevar un contador:

```
public class ReportesModelo {
    private static final String CARPETA_CLIENTES = "datos_clientes";
    public static final String CARPETA_AUTOS = "datos_autos";
    private static final String CARPETA REPUESTOS = "datos_repuestos";
    private static final String CARPETA SERVICIOS = "datos_servicios";

    private static int contadorClientes = 1;
    private static int contadorRepuestosUsados = 1;
    private static int contadorRepuestosCaros = 1;
    private static int contadorServiciosUsados = 1;
    private static int contadorAutosRepetidos = 1;

    public static String[][] obtenerTopServiciosUsados() {
        ServiciosModelo.Servicio[] servicios = ServiciosModelo.obtenerTodosServicios();

        //FILTRAR SOLO LOS SERVICIOS CON CONTADOR > 0
        int count = 0;
        for (ServiciosModelo.Servicio serv : servicios) {
            if (serv != null && serv.getContadorUsos() > 0) {
                count++;
            }
        }

        //ORDENAR POR CONTADOR DE USOS
        for (int i = 0; i < servicios.length - 1; i++) {
            for (int j = 0; j < servicios.length - i - 1; j++) {
                if (servicios[j] == null ||
                    (servicios[j+1] != null &&
                     servicios[j].getContadorUsos() < servicios[j+1].getContadorUsos())) {
                    ServiciosModelo.Servicio temp = servicios[j];
                    servicios[j] = servicios[j+1];
                    servicios[j+1] = temp;
                }
            }
        }
    }
}
```

RepuestosModelo

En esta clase hay getters y setters, para un funcionamiento correcto:

```
//GETTERS Y SETTERS
public int getId() { return id; }
public String getNombre() { return nombre; }
public void setNombre(String nombre) { this.nombre = nombre; }
public String getMarca() { return marca; }
public void setMarca(String marca) { this.marca = marca; }
public String getModelo() { return modelo; }
public void setModelo(String modelo) { this.modelo = modelo; }
public int getExistencias() { return existencias; }
public void setExistencias(int existencias) { this.existencias = existencias; }
public double getPrecio() { return precio; }
public void setPrecio(double precio) { this.precio = precio; }
public int getContadorUsos() { return contadorUsos; }
public void incrementarContadorUsos() { this.contadorUsos++; }
```

SerializadorModelo

En esta clase se encuentra la estructura para poder guardar archivos de tipo binario y que no se pierdan al cerrar y volver a abrir el programa.

```
//GUARDAR UN OBJETO EN UN ARCHIVO BINARIO
public boolean guardarDatos(String nombreArchivo, Object objeto) {
    String rutaCompleta = DIRECTORIO_DATOS + nombreArchivo;
    try (ObjectOutputStream oos = new ObjectOutputStream(
        new FileOutputStream(rutaCompleta))) {
        oos.writeObject(objeto);
        return true;
    } catch (IOException e) {
        return false;
    }
}
```

```
//CARGAR UN OBJETO DESDE UN ARCHIVO BINARIO
public Object cargarDatos(String nombreArchivo) {
    String rutaCompleta = DIRECTORIO_DATOS + nombreArchivo;
    File archivo = new File(rutaCompleta);
    if (!archivo.exists()) return null;

    try (ObjectInputStream ois = new ObjectInputStream(
        new FileInputStream(rutaCompleta))) {
        return ois.readObject();
    } catch (IOException | ClassNotFoundException e) {
        return null;
    }
}
```

Paquete Controlador

Este paquete recibe las entradas del usuario desde la vista, procesa estas solicitudes interactuando con el modelo para obtener y manipular los datos, para luego actualizar la vista con los resultados obtenidos.

InicioControlador

En esta clase se verifica el usuario con las credenciales ingresadas para llevar a su categoría:

```
switch (tipoUsuario) {
    case "ADMIN":
        registroModelo.setUsuarioActual(usuario);
        BitacoraModelo.registrarEvento(usuario, "Inicio de sesión", "Éxito", "Acceso como administrador");
        abrirMenuAdministrador();
        break;
    case "MECANICO":
        registroModelo.setUsuarioActual(usuario);
        BitacoraModelo.registrarEvento(usuario, "Inicio de sesión", "Éxito", "Acceso como mecánico");
        abrirMenuAdministrador();
        break;
    case "CLIENTE":
        registroModelo.setUsuarioActual(usuario);
        BitacoraModelo.registrarEvento(usuario, "Inicio de sesión", "Éxito", "Acceso como cliente");
        abrirMenuCliente();
        break;
    default:
        BitacoraModelo.registrarEvento(usuario, "Intento de inicio de sesión", "Error", "Credenciales incorrectas");
        JOptionPane.showMessageDialog(vista, "USUARIO O CONTRASEÑA INCORRECTOS", "ERROR", JOptionPane.ERROR_MESSAGE);
        vista.getCampoContraseña().setText("");
}
}
```

RegistroControlador

En esta clase se deben validar los campos para que se ingresen datos correctos:

```
private boolean validarCampos(String dpi, String nombre, String usuario, String contra) {
    if (dpi.isEmpty() || nombre.isEmpty() || usuario.isEmpty() || contra.isEmpty()) {
        BitacoraModelo.registrarEvento("Sistema", "Validación de datos", "Error", "Se debe llenar todos los campos");
        JOptionPane.showMessageDialog(vista, "DEBE LLENAR TODOS LOS CAMPOS", "ERROR", JOptionPane.ERROR_MESSAGE);
        return false;
    }

    if (usuario.equals(InicioModelo.ADMIN_USER) || usuario.equals(InicioModelo.MECANICO_USER)) {
        BitacoraModelo.registrarEvento(usuarioActual, "Nombre de Usuario", "Error", "Nombre de usuario no permitido");
        JOptionPane.showMessageDialog(vista, "NOMBRE DE USUARIO NO PERMITIDO", "ERROR", JOptionPane.ERROR_MESSAGE);
        return false;
    }

    return true;
}
```

BitacoraControlador

En esta clase se configuró la tabla para poder ingresar datos de forma correcta.

```
private void agregarRegistroATabla(BitacoraModelo.RegistroBitacora registro) {
    modelo.addRow(new Object[] {
        registro.getMarcaTemporal(),
        registro.getUsuario(),
        registro.getAccion(),
        registro.getResultado(),
        registro.getDetalles()
    });

    //AUTO-SCROLL AL ULTIMO REGISTRO
    vista.getTablaBitacora().scrollRectToVisible(
        vista.getTablaBitacora().getCellRect(modelo.getRowCount()-1, 0, true)
    );
}
```

RegistrarAutosControlador

En esta clase se realizó un código en específico para procesar la imagen ingresada:

```
private void procesarImagen(File file) {
    try {
        String fileName = file.getName().toLowerCase();
        if (!fileName.endsWith(".jpg") && !fileName.endsWith(".jpeg")) {
            BitacoraModelo.registrarEvento(usuarioActual, "Extensiones", "Error", "Se debe ingresar un a
            throw new IOException("SOLO SE PERMITEN ARCHIVOS .jpg o .jpeg");
        }

        BufferedImage originalImage = ImageIO.read(file);
        if (originalImage == null) {
            BitacoraModelo.registrarEvento(usuarioActual, "Imagen", "Error", "La imagen no es válida");
            throw new IOException("LA IMAGEN NO ES VALIDA");
        }

        //AJUSTAR IMAGEN AL PANEL
        Image scaledImage = originalImage.getScaledInstance(
            vista.getPanelImagen().getWidth(),
            vista.getPanelImagen().getHeight(),
            Image.SCALE_SMOOTH);
    }
}
```

AgregarRControlador

En esta clase se creó la funcionalidad para agregar repuestos de forma correcta con algunas restricciones:

```
if (id != -1) {
    BitacoraModelo.registrarEvento(usuarioActual, "Repuesto Agregado", "Éxito", "Repuesto agregado con ID: " + id);
    javax.swing.JOptionPane.showMessageDialog(vista, "REPUESTO AGREGADO CON ID: " + id);
    limpiarCampos();
    actualizarID();
} else {
    BitacoraModelo.registrarEvento(usuarioActual, "No se agregó el repuesto", "Error", "No se pudo agregar el repuesto");
    javax.swing.JOptionPane.showMessageDialog(vista, "NO SE PUDO AGREGAR EL REPUESTO", "ERROR", javax.swing.JOptionPane.ERROR_MESSAGE);
}
} catch (NumberFormatException e) {
    BitacoraModelo.registrarEvento(usuarioActual, "Validación de datos", "Error", "Datos invalidos");
    javax.swing.JOptionPane.showMessageDialog(vista, "INGRESE DATOS VALIDOS", "ERROR", javax.swing.JOptionPane.ERROR_MESSAGE);
}
```

ModificarRControlador

En esta clase se ha puesto la función para pedir un repuesto y verificar que las credenciales coincidan.

```
private void pedirIdRepuesto() {
    BitacoraModelo.registrarEvento(usuarioActual, "ID del repuesto", "Información", "Se pide el ID del repuesto");
    String idStr = JOptionPane.showInputDialog(vista, "INGRESE EL ID DEL REPUESTO:");
    if (idStr != null && !idStr.isEmpty()) {
        try {
            int id = Integer.parseInt(idStr);
            repuestoActual = RepuestosModelo.buscarRepuesto(id);

            if (repuestoActual != null) {
                //LLENAR CAMPOS CON DATOS ACTUALIZADOS
                vista.getCampoNombre().setText(repuestoActual.getNombre());
                vista.getCampoMarca().setText(repuestoActual.getMarca());
                vista.getCampoModelo().setText(repuestoActual.getModelo());
                vista.getCampoExistencias().setText(String.valueOf(repuestoActual.getExistencias()));
                vista.getCampoPrecio().setText(String.valueOf(repuestoActual.getPrecio()));
            } else {
                BitacoraModelo.registrarEvento(usuarioActual, "Validación de ID", "Error", "El ID ingresado no existe");
                JOptionPane.showMessageDialog(vista, "EL ID INGRESADO NO EXISTE", "ERROR", JOptionPane.ERROR_MESSAGE);
                regresar();
            }
        } catch (NumberFormatException e) {
            BitacoraModelo.registrarEvento(usuarioActual, "Validación de ID", "Error", "El DI ingresado es inválido");
            JOptionPane.showMessageDialog(vista, "ID INVALIDO", "ERROR", JOptionPane.ERROR_MESSAGE);
            regresar();
        }
    } else {
        regresar();
    }
}
```

VerControlador

Esta clase está configurada para en una tabla cargar los repuestos que han sido creados o modificado y actualizar los datos siempre:

```
private void cargarTablaRepuestos() {
    Repuesto[] repuestos = RepuestosModelo.obtenerTodosRepuestos();
    DefaultTableModel modelo = (DefaultTableModel) vista.getTabla().getModel();
    modelo.setRowCount(0); //LIMPIAR TABLA

    for (Repuesto repuesto : repuestos) {
        Object[] fila = {
            repuesto.getId(),
            repuesto.getNombre(),
            repuesto.getMarca(),
            repuesto.getModelo(),
            repuesto.getExistencias(),
            repuesto.getPrecio()
        };
        modelo.addRow(fila);
    }
}
```

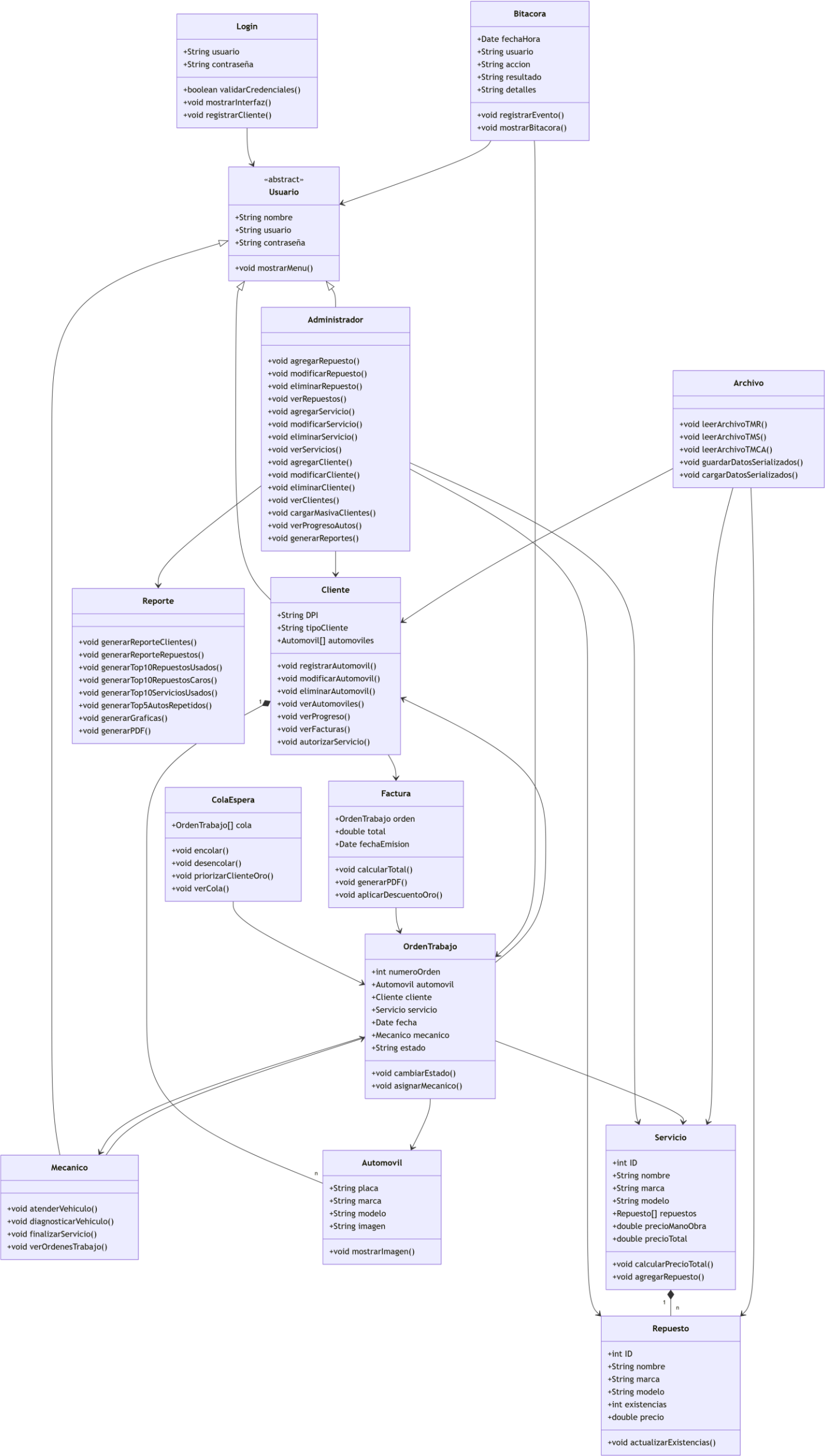
ProgresoControlador

En esta clase se requiere cargar varios datos, de vehículos y servicios, lo cual se realizó con el siguiente código:

```
private void cargarVehiculosCliente() {
    String[][] vehiculos = modelo.obtenerVehiculosCliente(usuarioActual);
    if (vehiculos != null) {
        vista.getListVehiculo().removeAllItems();
        for (String[] vehiculo : vehiculos) {
            if (vehiculo != null && vehiculo.length >= 3) {
                vista.getListVehiculo().addItem(vehiculo[0] + " - " + vehiculo[1] + " " + vehiculo[2]);
            }
        }
    } else {
        BitacoraModelo.registrarEvento(usuarioActual, "Vehiculos Registrados", "Advertencia", "No cuenta con");
        JOptionPane.showMessageDialog(vista, "NO SE ENCONTRARON VEHICULOS REGISTRADOS", "ADVERTENCIA", JOptionPane.INFORMATION_MESSAGE);
    }
}

private void cargarServiciosDisponibles() {
    ServiciosModelo.Servicio[] servicios = ServiciosModelo.obtenerTodosServicios();
    if (servicios != null) {
        vista.getListServicios().removeAllItems();
        for (ServiciosModelo.Servicio servicio : servicios) {
            if (servicio != null) {
                vista.getListServicios().addItem(servicio.getNombre());
            }
        }
    }
}
```


Diagrama de Clases



CONSIDERACIONES TÉCNICAS

¿Qué formato debe tener el archivo para ser procesado?

El archivo debe tener exactamente dos columnas, la primera de categorías y la segunda de valores numéricos. Donde la primera fila debe tener los encabezados.

¿Qué pasa si el archivo no cumple con el formato requerido?

El sistema mostrará un mensaje de error, indicando que el archivo no es válido. Asegúrese de que cuente con el formato indicado.

¿Puedo ordenar los datos sin cargar un archivo primero?

No, el sistema requiere que se cargue un archivo .tmr válido antes de habilitar las opciones de ordenamiento.

¿Dónde puedo ver el reporte PDF después de generarlo?

El sistema generará automáticamente un archivo PDF en la misma carpeta donde se ejecuta el programa.